



Project Report

Intern Name: Ahmed Islam

Department: Artificial Intelligence

Organization: IR Solutions

Designation: AI Intern

Week # 09

Submitted to: Asad Ahmed

Submission Date: 5-Sep-2025

Table of Contents

Project Report	1
Day 1 Report – Natural Language Processing (NLP)1. Theoretical Learning	3
a. Introduction to NLP and its Pipeline.....	3
b. Stemming vs Lemmatization	3
c. Text-to-Vector Conversion Techniques	4
d. Regular Expressions (Regex)	4
Day 2 Report – Generative AI & LangChain Basics	5
a. Introduction to Generative AI	5
b. Understanding ChatGPT’s Architecture.....	5
c. Introduction to LangChain	6
d. Focus Areas of the Day	6
Day 3 Report – Indexing, RAG, and Structured Outputs.....	7
a. Indexing in LangChain	7
b. Retrieval-Augmented Generation (RAG)	7
c. Structured vs Unstructured Outputs	8
Day 4 Report – Output Parsers & Chains in LangChain	8
a. Output Parsers in LangChain	8
b. Chains in LangChain	9
Practical Implementation	9
Task 1:	9
Task 2:	10

Day 1 Report – Natural Language Processing (NLP)1.

Theoretical Learning

a. Introduction to NLP and its Pipeline

- **Definition:**

Natural Language Processing (NLP) is a branch of Artificial Intelligence that enables computers to understand, interpret, and generate human language.

- **NLP Pipeline:**

The NLP pipeline consists of several stages to process text data before using it in downstream tasks like classification, translation, or chatbots.

Common steps include:

1. **Text Acquisition** – Collecting raw text data.
2. **Text Cleaning/Preprocessing** – Removing unwanted characters, stopwords, etc.
3. **Tokenization** – Splitting text into words or sentences.
4. **Stemming/Lemmatization** – Reducing words to their root/base form.
5. **Vectorization** – Converting text into numerical representation.
6. **Modeling** – Applying ML/DL models to perform NLP tasks.

b. Stemming vs Lemmatization

- **Stemming:**

- A rule-based technique that chops off prefixes/suffixes to reduce words to their stem form.
- Example: “*studies*” → “*studi*”, “*running*” → “*run*”.
- Fast but sometimes produces non-meaningful words.

- **Lemmatization:**

- A more sophisticated approach that reduces words to their meaningful root (lemma) using vocabulary and grammar.
- Example: “*studies*” → “*study*”, “*better*” → “*good*”.

- Slower but more accurate.

c. Text-to-Vector Conversion Techniques

To process text with machine learning models, words must be converted into numbers. Three common approaches are:

1. **Bag of Words (BoW):**

- Represents text as a vector of word counts.
- Simple but ignores context and order.

2. **TF-IDF (Term Frequency – Inverse Document Frequency):**

- Extends BoW by weighing words according to their importance in a document relative to a collection of documents.
- Helps reduce the impact of common words like “the,” “and.”

3. **Word Embeddings:**

- Dense vector representations capturing semantic meaning.
- Words with similar meanings have vectors close to each other.
- Examples: **Word2Vec, GloVe, FastText.**

d. Regular Expressions (Regex)

- **Definition:** Regex is a sequence of characters that defines a search pattern, mainly used for text extraction, validation, and manipulation.
- **Examples of use cases:**
 - Extracting **emails**: `\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b`
 - Extracting **phone numbers**: `\d{3}-\d{3}-\d{4}`
 - Finding **dates, keywords, or patterns** in text.

Day 2 Report – Generative AI & LangChain Basics

a. Introduction to Generative AI

- **Definition:**

Generative AI is a subset of artificial intelligence that can create new content such as text, images, audio, or code by learning patterns from existing data.

- **Working Principle:**

- Uses large neural networks trained on massive datasets.
- Predicts the next element in a sequence (e.g., the next word in a sentence).
- Learns contextual relationships, enabling creative and coherent outputs.

- **Applications:**

- Chatbots (e.g., ChatGPT)
- Text summarization
- Code generation
- Image creation (DALL-E, Stable Diffusion)

b. Understanding ChatGPT's Architecture

- **Transformer Architecture:**

ChatGPT is built on the **Transformer** model, which uses *attention mechanisms* to process sequences of words.

- **Encoder-Decoder Models:**

- **Encoder:** Reads and understands the input text, converting it into hidden representations.
- **Decoder:** Uses these representations to generate relevant output word by word.

- **ChatGPT Variant:**

- Based on **Decoder-only Transformers (GPT)**.
- Focuses entirely on predicting the next word in context.
- Trained on large-scale internet data using **unsupervised learning + fine-tuning**.

c. Introduction to LangChain

- **What is LangChain?**

A Python framework designed for building powerful applications that use large language models (LLMs).

- **Why use it?**

- Provides tools for prompt management.
- Allows chaining of components (prompt → model → output).
- Supports memory for conversational apps.
- Integrates retrieval systems for RAG.

- **Six Core Components of LangChain:**

1. **Prompt** – Input instructions given to the model.
2. **Model** – The LLM being used (e.g., GPT, Hugging Face models).
3. **Chain** – A sequence of calls connecting prompts, models, and outputs.
4. **Memory** – Stores context across multiple user interactions.
5. **Index** – Manages documents and embeddings for retrieval.
6. **Agents** – Decision-making entities that use tools and models dynamically.

d. Focus Areas of the Day

- Explored **Prompts** in detail: how wording affects LLM responses.
- Understood **Models** and their role in generating outputs.
- Learned how LangChain simplifies prompt → model → output workflows.

Day 3 Report – Indexing, RAG, and Structured Outputs

a. Indexing in LangChain

- **Definition:**

Indexing in LangChain is the process of structuring and storing documents so they can be efficiently retrieved later.

- **Why it matters:**

- Allows **document search** and **retrieval**.
- Converts text into **embeddings** (vector representations).
- Stores these embeddings in a **vector database** for quick similarity search.

- **Workflow:**

1. Load documents (PDFs, text files, etc.).
2. Split documents into chunks.
3. Convert chunks into embeddings.
4. Store embeddings in a vector database (e.g., FAISS, Pinecone).
5. Retrieve relevant chunks when a query is asked.

b. Retrieval-Augmented Generation (RAG)

- **Concept:**

RAG combines **retrieval** (fetching relevant information) with **generation** (LLM response).

- **Why RAG is important:**

- Reduces hallucinations in LLMs by grounding answers in external data.
- Makes LLMs context-aware.
- Enables domain-specific applications (e.g., *chat with your PDF*, legal doc summarization).

- **RAG Pipeline Steps:**

1. **Document Retrieval** – Fetch relevant text chunks from an index.

2. **Chunking & Embedding** – Break text into smaller pieces, convert into vectors.
3. **Vector Search** – Retrieve the most similar chunks to the query.
4. **LLM Generation** – Combine retrieved data with the user query to generate an answer.

c. Structured vs Unstructured Outputs

- **Unstructured Output:**
 - Raw text directly from the model.
 - Example: “The answer is 42.”
- **Structured Output:**
 - Well-formatted results like JSON, dictionaries, or objects.
 - Easier to parse, validate, and use in applications.
- **TypedDictionary & Pydantic Schema:**
 - **TypedDict:** Defines a dictionary with fixed keys and types (from Python typing).
 - **Pydantic Schema:** Defines data models with validation. Often used with LLM outputs to ensure correctness.

Day 4 Report – Output Parsers & Chains in LangChain

a. Output Parsers in LangChain

- **Definition:**

Output Parsers in LangChain transform the raw text generated by an LLM into structured and usable formats.
- **Why important?**
 - LLMs usually return unstructured text.
 - Applications often require structured formats (JSON, lists, dictionaries).
 - Output Parsers enforce format correctness.

- **Types of Output Parsers:**

1. **Simple Output Parser:** Converts text into plain strings or numbers.
2. **List Output Parser:** Converts text into Python lists.
3. **JSON Output Parser:** Converts text into valid JSON objects.
4. **Pydantic Output Parser:** Validates outputs against a defined schema.
5. **Custom Parsers:** Created when more control is needed.

b. Chains in LangChain

- **Definition:**

A **Chain** is a sequence of calls where the output of one step becomes the input of the next.

- **Why Chains are useful?**

- Automate multi-step workflows.
- Connect prompts, models, parsers, and tools into one pipeline.
- Make LLM applications modular and reusable.

- **Types of Chains:**

1. **Simple LLM Chain:** One prompt → one model → one output.
2. **Sequential Chain:** Multiple LLM calls executed step by step.
3. **Router Chain:** Routes input to different chains based on conditions.
4. **Retrieval Chain:** Combines retrieval (from vector DB) with LLM response.
5. **Custom Chains:** User-defined workflows.

Practical Implementation

Task 1:

For hands-on practice, I:

- Implemented **Stemming and Lemmatization** using NLTK and spaCy.

- Practiced different text-to-vector conversion methods on sample datasets.
- Explored the **Regex library in Python** to extract information from text using different matching patterns.
- Used **Google Colab** for implementation and experimentation.

🔗 **Colab Notebook:**

<https://colab.research.google.com/drive/1ohb6XwxWthhFegkeeNjfdakAalUcb--l?usp=sharing>

<https://colab.research.google.com/drive/1-7H4g5za5SuQ2wpgKqOmx6XkqkRao4wh?usp=sharing>

Task 2:

For practice, I:

- Implemented **Indexing** using vector databases.
- Built a **RAG app** that allows users to chat with PDFs by specifying their path.
- Enhanced the RAG system to **extract data from multiple PDFs simultaneously**, ensuring that responses are strictly limited to the information present in those documents (no hallucinations).
- Used **Google Colab** for implementation.

🔗 **Colab Notebook (RAG App):** <https://colab.research.google.com/drive/18PxfAV--kJC0BG5wCNvT7wj7aKhDaVOR?usp=sharing>